VITCC ENTERPRISE LIBRARY MANAGEMENT SYSTEM

Dual-Paradigm Relational (Oracle 21c) and Graph (Neo4j AuraDB) Database Application Project

Course Project Assessment (DA-3) covering Schema Normalization, PL/SQL Automation, Graph Traversal & Role-Based Access Control

COURSE & FACULTY DETAILS

Course Code: BACSE202

Course Name: Database Systems

Class Slot: F1+TF1

Faculty Guide: Dr. R. Priyadarshini

Department: Department of Computer Science & Engineering

Institution: VIT Chennai Campus

SUBMITTED BY (PROJECT TEAM)

KARTHIK VALLIAPPAN C (25BAI1051)
System Architecture, Neo4j Graph Model & Web Controllers

NITHIN RANGASWAMY (25BAI1210)
Relational Schema, Oracle DDL/DML, Constraints & Auditing

NATRAJ (25BAI1079)
PL/SQL Programming, Stored Procedures, Functions & Triggers

Presentation Roadmap & Topics Covered

SECTION 1
Introduction & Architecture

- Problem Statement & Objectives
- Proposed Dual-Database Solution
- 4-Tier Architecture & Tech Stack
- Core Modules & Data Interaction

SECTION 2
Database Design & Schema

- Entity-Relationship (ER) Diagram
- Relational Mapping & Normalization (BCNF)
- Table DDL & Integrity Constraints
- Sample Records & Referential Integrity

SECTION 3
SQL & Database Programming

- Feature-to-Query Mapping
- DDL, DML & Role-Based DCL Security
- Analytical Subqueries & Complex Joins
- PL/SQL Triggers, Cursors & Procedures

SECTION 4
Implementation & Results

- Application Workflow & Code Structure
- Working Application Screenshots (8 views)
- Role-Based Access Control & Innovations
- Build Steps, GitHub Repository & Conclusion

Problem Statement, Objectives & Solution Scope

PROBLEM STATEMENT

• Traditional relational library management systems struggle with recursive multi-table joins when discovering multi-degree book connections.

• Students lack structured academic reading roadmaps showing prerequisite foundational books before advanced texts.

• Manual circulation desk verification often fails to enforce patron loan limits (3 books) and calculate overdue penalties reliably.

• Borrowing histories are exposed across patrons without proper role-based privacy masking.

PROPOSED SOLUTION & SCOPE

• Phased Database Migration: Phase 1 utilized Oracle 21c for strict relational modeling, while Phase 2 (DA-3) completely migrated the backend to Neo4j AuraDB to unlock advanced graph traversals and recommendations.

• Role-Based Access Control (RBAC): Provides four distinct experiences for Admin, Librarian, Staff (Faculty), and Students.

• Automated Circulation Rules: Real-time 3-book borrowing quota enforcement and daily Rs. 10/day overdue fine calculation.

• Interactive Graph Capabilities: Real-time knowledge graph visualizer, shortest-path degrees of separation, and DAG prerequisite learning tracks.

TECHNOLOGY STACK SUMMARY

Web Framework

Python 3.12, Flask 3.1, Flask-Session, Flask-WTF CSRF

Relational Engine

Oracle Database 21c (Tables, Constraints, PL/SQL)

Graph Engine

Neo4j AuraDB v5+ (Cypher, Graph Traversal Algorithms)

Frontend UI

HTML5, Vanilla CSS, Plus Jakarta Sans, JetBrains Mono

Graph Canvas

Vis.js Network v9.1.2 (Interactive Physics Rendering)

Verification

7 End-to-End Automated Verification Suites (100% Pass)

Overall System Architecture & Multi-Tier Data Interaction

Tier 1: Client & Presentation Layer
Web Browser UI & Interactive Graph Canvas

- Responsive Web Portal implemented in HTML5 and clean Vanilla CSS (Modern Slate Theme).
- Vis.js physics-driven canvas rendering interactive knowledge graphs and prerequisite roadmaps.
- Role-specific UI views tailored for Admin, Librarian, Faculty/Staff, and Students.

Tier 2: Application Controller & Security Layer
Flask WSGI Middleware & RBAC Controllers

- Central request router managing authentication, catalog browsing, loan processing, and graph APIs.
- Flask-WTF CSRF token validation on all mutable operations (loan issue, returns, password updates).
- 10-minute idle session timeout with automatic server-side cleanup and institutional email checks.

Tier 3: Data Access & Business Logic Layer
Parameterized Database Drivers & Controllers

- Neo4j Bolt Protocol (neo4j+s://) executing optimized Cypher shortestPath() and prerequisite queries.
- Dynamic patron privacy masking to anonymize fellow student loan requests on shared canvases.

Tier 4: Enterprise Storage Tier
Phase 1 (Legacy Oracle) & Phase 2 (Live Neo4j Graph)

- Oracle 21c Database: 10 normalized tables in BCNF, foreign key cascades, triggers, and views.
- Neo4j AuraDB: 9 node entity types, 7 relationship edges, and 12 schema uniqueness constraints.
- Automated quota enforcement, copy status synchronization, and audit logging.

Conceptual Data Model — Entity-Relationship (ER) Diagram

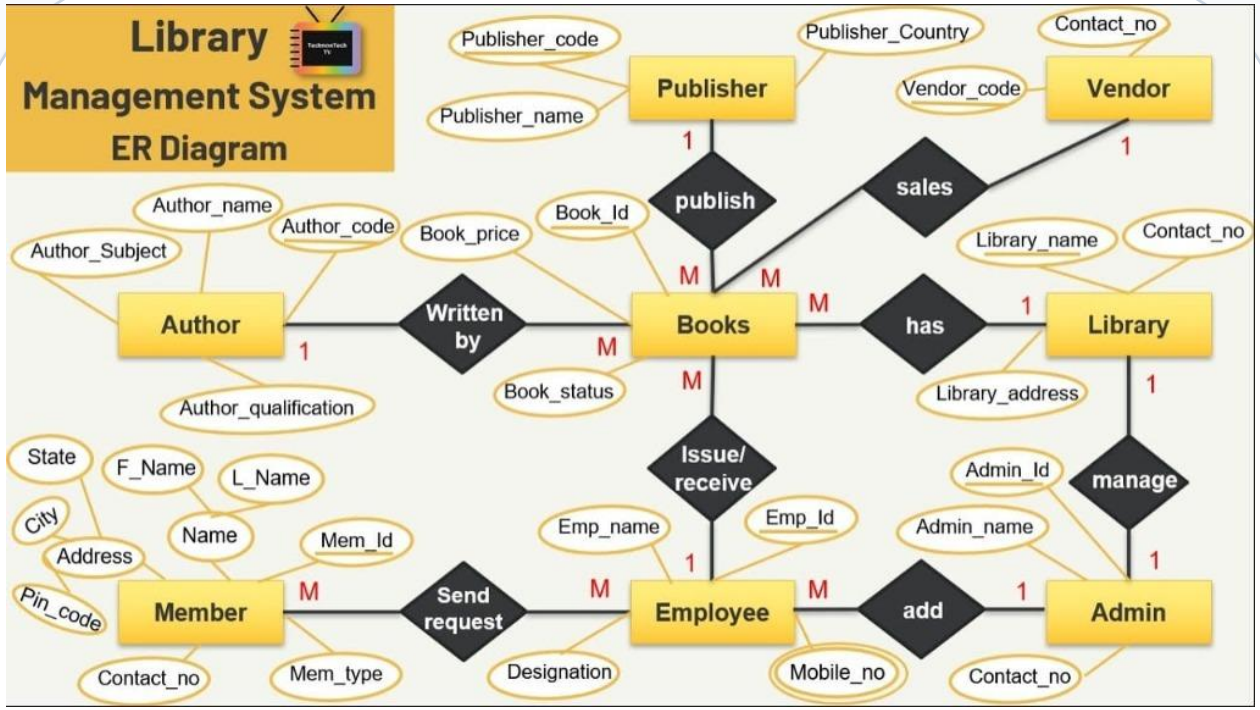


Figure: Conceptual Entity-Relationship (ER) Diagram of the Library Management System

ENTITY & CARDINALITY MAPPING

1:N Publisher to Books

Each publisher publishes multiple titles; every book is linked to a single registered publisher.

M:N Books to Authors

A book may have co-authors, and authors write multiple books. Mapped into the BOOK_AUTHORS relationship table.

1:N Book to Copies

Separates abstract bibliographic titles from physical inventory tracking copies (status: Available, Borrowed, Reserved).

1:N User to Loans

Patrons record historical loans over time, strictly capped at 3 active concurrent borrowed books.

1:1 Loan to Fines

Overdue book returns automatically generate a financial disciplinary record (Rs. 10 per overdue day).

Relational Normalization

Conceptual Member, Employee, and Admin roles are unified into the USERS entity with Role-Based Access Control.

Relational Schema & Functional Dependency Normalization (BCNF)

RELATIONAL SCHEMA SPECIFICATIONS

CATEGORIES (category_id [PK], category_name, description)

PUBLISHERS (publisher_id [PK], name, address, phone, email)

AUTHORS (author_id [PK], first_name, last_name, bio)

BOOKS (book_id [PK], title, isbn, publication_year, category_id [FK], publisher_id [FK])

BOOK_AUTHORS (book_id [PK, FK], author_id [PK, FK])

BOOK_COPIES (copy_id [PK], book_id [FK], status, acquisition_date)

USERS (user_id [PK], username, password_hash, email, role, account_status)

LOANS (loan_id [PK], copy_id [FK], user_id [FK], issue_date, due_date, return_date, issued_by [FK])

LOAN_REQUESTS (request_id [PK], user_id [FK], book_id [FK], request_date, status, copy_id [FK])

FINES (fine_id [PK], loan_id [FK], fine_amount, payment_status, payment_date, collected_by [FK])

NORMALIZATION DECOMPOSITION

1NF (First Normal Form)

All table attributes contain atomic values. Repeated author and copy collections are separated into distinct tables.

2NF (Second Normal Form)

Eliminated all partial key dependencies. In BOOK_AUTHORS, the composite key {book_id, author_id} determines full relationship status.

3NF (Third Normal Form)

Eliminated all transitive functional dependencies (e.g., category_name is determined exclusively by category_id, not transitively by book_id).

BCNF (Boyce-Codd Normal Form)

Every determinant is a candidate key. Protects against update, insertion, and deletion anomalies across the loan lifecycle.

Table Creation & Comprehensive Integrity Constraints (DDL)

```
-- Table Creation with Integrity Constraints (Oracle SQL)
CREATE TABLE USERS (
    user_id        NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    username       VARCHAR2(50) NOT NULL UNIQUE,
    email          VARCHAR2(100) NOT NULL UNIQUE,
    password_hash  VARCHAR2(255) NOT NULL,
    role           VARCHAR2(20) DEFAULT 'Student' NOT NULL,
    account_status VARCHAR2(20) DEFAULT 'Active' NOT NULL,
    CONSTRAINT chk_user_role CHECK (role IN ('Admin', 'Librarian', 'Staff', 'Student')),
    CONSTRAINT chk_user_status CHECK (account_status IN ('Active', 'Suspended'))
);

CREATE TABLE BOOK_COPIES (
    copy_id NUMBER PRIMARY KEY,
    book_id NUMBER NOT NULL REFERENCES BOOKS(book_id) ON DELETE CASCADE,
    status  VARCHAR2(20) DEFAULT 'Available' NOT NULL,
    CONSTRAINT chk_copy_status CHECK (status IN ('Available', 'Borrowed', 'Reserved',
'Maintenance'))
);

CREATE TABLE LOANS (
    loan_id     NUMBER PRIMARY KEY,
    copy_id     NUMBER NOT NULL REFERENCES BOOK_COPIES(copy_id),
    user_id     NUMBER NOT NULL REFERENCES USERS(user_id) ON DELETE CASCADE,
    issue_date  DATE DEFAULT SYSDATE NOT NULL,
    due_date    DATE NOT NULL,
    return_date DATE,
    CONSTRAINT chk_loan_dates CHECK (due_date >= issue_date)
);
```

INTEGRITY CONSTRAINTS IMPLEMENTED

PRIMARY KEY Constraints

Enforces entity integrity (user_id, book_id, loan_id). Prohibits duplicate or NULL primary records.

FOREIGN KEY & CASCADES

Maintains referential integrity. Removing a book cascades to dependent copies (ON DELETE CASCADE), preventing dangling references.

CHECK Constraints

chk_user_role restricts accounts to authorized roles; chk_copy_status ensures valid inventory states; chk_loan_dates guarantees due_date >= issue_date.

UNIQUE Constraints

Prevents duplicate ISBN registrations, identical usernames, and duplicate institutional email addresses.

NOT NULL & DEFAULT

Mandates critical metadata (password_hash, email) while initializing default statuses ('Available', 'Active', 'Unpaid').

Sample Data Records & Referential Integrity Verification

SAMPLE ENTITY RECORDS POPULATED IN DATABASE

Table	Identifier (PK)	Attribute 1	Attribute 2	Status / Role	Foreign Key Ref
USERS	user_id = 1	admin	admin@vit.edu	Role: Admin	Account: Active
BOOKS	book_id = 2	Clean Code: Agile Craft	ISBN: 978-0132350884	Pub Year: 2008	Category: CS (ID: 1)
LOANS	loan_id = 14	copy_id = 2	user_id = 6 (student)	Issue: 2026-03-01	Due: 2026-03-15 (Overdue)

REFERENTIAL INTEGRITY ENFORCEMENT

- Foreign Key Cascading: Deleting a category safely cascades to dependent book records or blocks deletion if active loans exist.
- Copy Status Synchronization: Physical copies transition automatically to 'Borrowed' upon loan issue and 'Available' upon check-in.
- Borrower Anonymization on Account Deletion: Historical loans retain record integrity (borrower_id = 0) to maintain statistical ledgers.

CONSTRAINT REJECTION TESTING (ERROR-PROOFING)

- Duplicate ISBN Rejection: ORA-00001 unique constraint violation raised when attempting to re-register an existing ISBN.
- Borrowing Quota Rejection: PL/SQL trigger and Flask controllers immediately reject a 4th concurrent loan request.
- Disciplinary Fine Locking: Loan issuance is blocked when outstanding disciplinary fines exist on a patron's record.

SQL Queries Mapped to Application Features

Catalog & Category Search (/catalog, /category/<id>)

```
SELECT b.book_id, b.title, b.isbn, b.publication_year, c.category_name,
       COUNT(cp.copy_id) AS total_copies,
       SUM(CASE WHEN cp.status = 'Available' THEN 1 ELSE 0 END) AS available_copies
FROM BOOKS b
JOIN CATEGORIES c ON b.category_id = c.category_id
LEFT JOIN BOOK_COPIES cp ON b.book_id = cp.book_id
WHERE LOWER(b.title) LIKE :query OR REPLACE(b.isbn, '-', '') LIKE :isbn_query
GROUP BY b.book_id, b.title, b.isbn, b.publication_year, c.category_name;
```

Operational Purpose: Enables fast searching across titles, authors, and hyphenated/unhyphenated ISBNs with real-time copy availability counts.

Issue Loan Transaction (/issue_loan)

```
-- Atomic Loan Creation and Copy Status Lock
INSERT INTO LOANS (loan_id, copy_id, user_id, issue_date, due_date, issued_by)
VALUES (seq_loan_id.NEXTVAL, :copy_id, :user_id, SYSDATE, SYSDATE + 14, :admin_id);

UPDATE BOOK_COPIES
SET status = 'Borrowed'
WHERE copy_id = :copy_id;
```

Operational Purpose: Guarantees that a physical copy cannot be concurrently double-issued to another patron at the circulation desk.

DDL, DML, and Role-Based DCL Security Commands

DDL (DATA DEFINITION)

CREATE TABLE / INDEX

```
CREATE TABLE LOAN_REQUESTS (
  request_id NUMBER PRIMARY KEY,
  user_id NUMBER REFERENCES USERS,
  book_id NUMBER REFERENCES BOOKS,
  status VARCHAR2(20) DEFAULT 'Pending'
);
```

ALTER TABLE CONSTRAINT

```
ALTER TABLE FINES ADD CONSTRAINT chk_fine_amt
CHECK (fine_amount >= 0);
```

CREATE UNIQUE INDEX

```
CREATE UNIQUE INDEX idx_isbn ON BOOKS(isbn);
```

DML (DATA MANIPULATION)

INSERT DATA

```
INSERT INTO BOOK_COPIES (copy_id, book_id, status)
VALUES (seq_copy.NEXTVAL, 2, 'Available');
```

UPDATE TRANSACTION

```
UPDATE FINES SET payment_status = 'Paid',
payment_date = SYSDATE WHERE fine_id = :fid;
```

DELETE CONDITIONAL

```
DELETE FROM LOAN_REQUESTS
WHERE status = 'Rejected' AND request_date < SYSDATE-
30;
```

DCL (DATA CONTROL)

ROLE PROVISIONING

```
CREATE ROLE LIBRARIAN_ROLE;
CREATE ROLE STUDENT_ROLE;
```

GRANT PRIVILEGES

```
GRANT SELECT, INSERT, UPDATE ON LOANS
TO LIBRARIAN_ROLE;
GRANT SELECT ON BOOKS TO STUDENT_ROLE;
```

REVOKE RESTRICTIONS

```
REVOKE DELETE ON BOOKS FROM STUDENT_ROLE;
REVOKE UPDATE ON USERS FROM LIBRARIAN_ROLE;
```

Single-Row, Multiple-Row & Correlated Subqueries

1. Single-Row Subquery (Scalar Comparison)

```
SELECT b.book_id, b.title, b.publication_year
FROM BOOKS b
WHERE b.publication_year > (
    SELECT AVG(publication_year)
    FROM BOOKS
    WHERE category_id = b.category_id
);
```

Operational Analytics: Identifies books published more recently than the average publication vintage of their respective subject category.

2. Multiple-Row Subquery with IN Operator

```
SELECT u.user_id, u.username, u.email
FROM USERS u
WHERE u.user_id IN (
    SELECT DISTINCT l.user_id
    FROM LOANS l
    JOIN FINES f ON l.loan_id = f.loan_id
    WHERE f.payment_status = 'Unpaid'
);
```

Operational Analytics: Flags all patrons with outstanding overdue financial penalties to restrict borrowing permissions at checkout.

3. Correlated Subquery with EXISTS Operator

```
SELECT b.book_id, b.title
FROM BOOKS b
WHERE EXISTS (
    SELECT 1
    FROM BOOK_COPIES cp
    JOIN LOANS l ON cp.copy_id = l.copy_id
    WHERE cp.book_id = b.book_id
    GROUP BY cp.copy_id
    HAVING COUNT(l.loan_id) >= 3
);
```

Operational Analytics: Discovers 'High Circulation Velocity' titles where copies have been actively checked out 3 or more times.

SQL Functions — Aggregate, String, Date & Analytical Math

AGGREGATE FUNCTIONS & GROUP BY

COUNT(*), COUNT(DISTINCT l.user_id)

Measures total circulation transactions and distinct active patron engagement across semesters.

SUM(f.fine_amount)

Computes cumulative institutional penalty revenue and outstanding unpaid receivables.

AVG(MONTHS_BETWEEN(SYSDATE, b.acquisition_date))

Computes the mean retention age of catalog acquisitions for inventory deprecation.

MIN(due_date), MAX(due_date)

Identifies the earliest pending return and latest active return deadlines across campus.

GROUP BY ... HAVING COUNT(*) > 5

Filters prolific authors and high-density categories exceeding borrowing thresholds.

SCALAR, STRING & DATE FUNCTIONS

TO_CHAR(issue_date, 'YYYY-MM-DD HH24:MI')

Standardizes temporal audit timestamps across disparate client locales.

TRUNC(SYSDATE) - TRUNC(due_date)

Evaluates precise elapsed calendar days for overdue fine calculations.

UPPER(u.username), LOWER(u.email)

Normalizes patron authentication credentials to ensure case-insensitive identity matching.

REPLACE(b.isbn, '-', '')

Strips punctuation from ISBN strings for seamless barcode and query pattern matching.

NVL(return_date, 'Active Loan')

Handles NULL return dates gracefully in patron financial ledgers and dashboard reports.

Multi-Table Relational Joins & Abstract Database Views

MULTI-TABLE RELATIONAL JOINS (INNER & LEFT OUTER)

```
-- 5-Table Master Circulation Ledger Join
SELECT l.loan_id, u.username, u.role, b.title, c.category_name, cp.copy_id, l.issue_date, l.due_date, l.return_date
FROM LOANS l
INNER JOIN USERS u        ON l.user_id = u.user_id
INNER JOIN BOOK_COPIES cp ON l.copy_id = cp.copy_id
INNER JOIN BOOKS b        ON cp.book_id = b.book_id
INNER JOIN CATEGORIES c   ON b.category_id = c.category_id
LEFT OUTER JOIN FINES f   ON l.loan_id = f.loan_id;
```

DATABASE VIEWS FOR APPLICATION REPORTING & SECURITY MASKING

```
-- Abstract View: Masks sensitive password hashes and pre-aggregates patron financial exposure
CREATE OR REPLACE VIEW VIEW_PATRON_FINANCIAL_SUMMARY AS
SELECT u.user_id, u.username, u.email, u.role, u.account_status,
       COUNT(DISTINCT l.loan_id) AS total_borrowings,
       NVL(SUM(CASE WHEN f.payment_status = 'Unpaid' THEN f.fine_amount ELSE 0 END), 0) AS total_unpaid_fines
FROM USERS u
LEFT JOIN LOANS l ON u.user_id = l.user_id
LEFT JOIN FINES f ON l.loan_id = f.loan_id
GROUP BY u.user_id, u.username, u.email, u.role, u.account_status;
```

Database Triggers, Explicit Cursors & Automated Auditing

PL/SQL TRIGGER: QUOTA ENFORCEMENT

```
CREATE OR REPLACE TRIGGER trg_check_borrow_quota
BEFORE INSERT ON LOANS
FOR EACH ROW
DECLARE
    v_active_loans NUMBER;
BEGIN
    -- Count currently active unreturned books
    SELECT COUNT(*)
    INTO v_active_loans
    FROM LOANS
    WHERE user_id = :NEW.user_id
      AND return_date IS NULL;

    IF v_active_loans >= 3 THEN
        RAISE_APPLICATION_ERROR(-20001,
          'Policy Violation: Patron has reached max 3-book quota.');
    END IF;
END;
```

EXPLICIT CURSOR: OVERDUE FINE BATCH

```
DECLARE
    CURSOR c_overdue IS
        SELECT loan_id, due_date, TRUNC(SYSDATE) - TRUNC(due_date) AS days_late
        FROM LOANS
        WHERE return_date IS NULL
          AND due_date < SYSDATE;
    r_ov c_overdue%ROWTYPE;
BEGIN
    OPEN c_overdue;
    LOOP
        FETCH c_overdue INTO r_ov;
        EXIT WHEN c_overdue%NOTFOUND;

        -- Auto-calculate Rs. 10/day fine
        UPDATE FINES
        SET fine_amount = r_ov.days_late * 10.0
        WHERE loan_id = r_ov.loan_id
          AND payment_status = 'Unpaid';
    END LOOP;
    CLOSE c_overdue;
    COMMIT;
END;
```

Stored Procedures, Functions & Robust Exception Handling

ENTERPRISE PL/SQL STORED PROCEDURE WITH PARAMETERS & TRANSACTION CONTROL

```
CREATE OR REPLACE PROCEDURE sp_issue_book_loan (
    p_user_id   IN  NUMBER,
    p_copy_id   IN  NUMBER,
    p_admin_id  IN  NUMBER,
    p_loan_id   OUT NUMBER,
    p_status    OUT VARCHAR2
) AS
    v_copy_status   VARCHAR2(20);
    v_unpaid_fines  NUMBER;
    ex_copy_busy    EXCEPTION;
    ex_unpaid_fines EXCEPTION;
BEGIN
    -- Step 1: Verify physical inventory availability
    SELECT status INTO v_copy_status FROM BOOK_COPIES WHERE copy_id = p_copy_id FOR UPDATE;
    IF v_copy_status != 'Available' THEN RAISE ex_copy_busy; END IF;

    -- Step 2: Verify zero outstanding financial penalties
    SELECT NVL(SUM(fine_amount), 0) INTO v_unpaid_fines
    FROM FINES f JOIN LOANS l ON f.loan_id = l.loan_id
    WHERE l.user_id = p_user_id AND f.payment_status = 'Unpaid';
    IF v_unpaid_fines > 0 THEN RAISE ex_unpaid_fines; END IF;

    -- Step 3: Insert loan and flip copy status atomically
    INSERT INTO LOANS (copy_id, user_id, issue_date, due_date, issued_by)
    VALUES (p_copy_id, p_user_id, SYSDATE, SYSDATE + 14, p_admin_id)
    RETURNING loan_id INTO p_loan_id;

    UPDATE BOOK_COPIES SET status = 'Borrowed' WHERE copy_id = p_copy_id;

    COMMIT;
    p_status := 'SUCCESS';
EXCEPTION
    WHEN ex_copy_busy THEN
        ROLLBACK;
        p_status := 'ERROR: Physical book copy is currently reserved or checked out.';
    WHEN ex_unpaid_fines THEN
        ROLLBACK;
        p_status := 'ERROR: Patron has outstanding unpaid disciplinary fines.';
    WHEN OTHERS THEN
```

Application Workflow & Modular Code Architecture

PROJECT DIRECTORY TREE

```
DBS DA/
├── app.py                # Core Flask app (24 routes, RBAC, Cypher/SQL)
├── seed_all_data.py      # Neo4j schema constraints & sample data seed
├── requirements.txt      # Python dependencies (Flask, Neo4j, WTForms)
├── .env.example          # Database connection template
├── .gitignore            # Clean Git exclusions
├── README.md             # Comprehensive architecture documentation
├── templates/            # 24 Jinja2 HTML5 Professional Views
│   ├── base.html         # Sidebar, topbar, navigation & CSRF token
│   ├── dashboard.html    # Role-based dashboard (Admin/Staff/Student)
│   ├── catalog.html      # Category directory with copy aggregations
│   ├── issue_loan.html   # Physical loan creation desk
│   ├── graph_visualizer  # Interactive Vis.js live knowledge graph
│   ├── book_bridge.html  # Shortest path traversal explorer
│   └── learning_paths    # Acyclic prerequisite roadmaps
└── scratch/              # Verification suites (phase1..phase6, test runners)
```

HTTP REQUEST EXECUTION LIFECYCLE

Step 1: Inbound HTTP Request

Client browser triggers route (e.g. POST /issue_loan) with form payload and CSRF token.

Step 2: Security & Session Middleware

Verifies active session in filesystem, validates 10-minute idle activity timestamp, and decrypts CSRF token.

Step 3: Role-Based Authorization

@role_required(['Admin', 'Librarian']) decorator checks user role; redirects unauthorized roles to 403 Access Denied.

Step 4: Query Execution & Business Logic

Cypher query or PL/SQL procedure invoked via database driver session with parameterized injection protection.

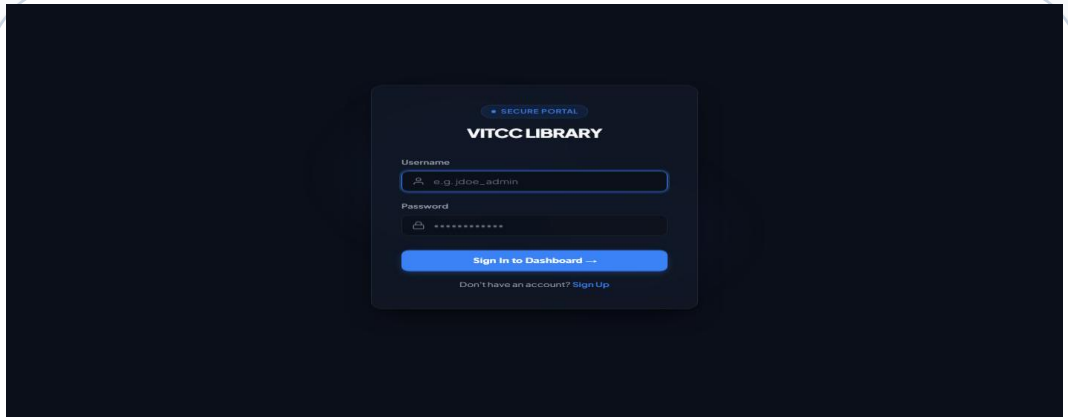
Step 5: Dynamic View Rendering

Jinja2 compiles response template with DateWrapper objects and privacy-masked attributes.

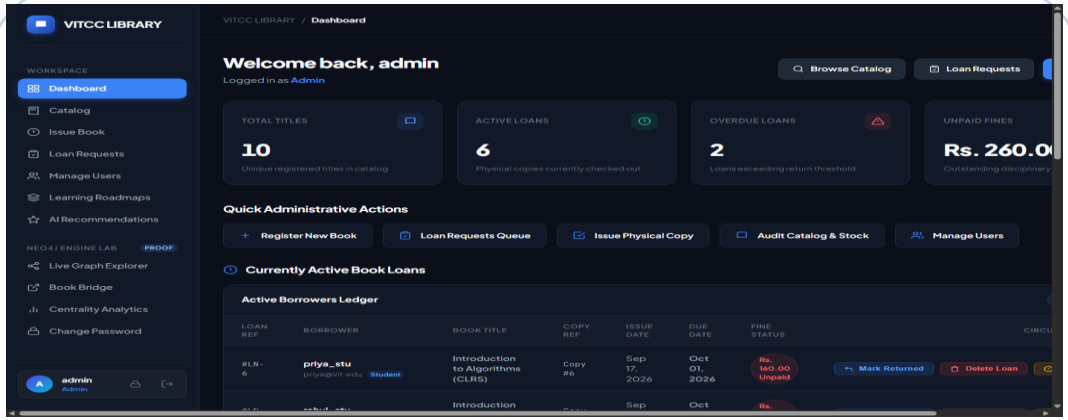
Step 6: Client DOM Update

Rendered HTML/CSS returned with clean professional visuals and interactive Vis.js graph physics.

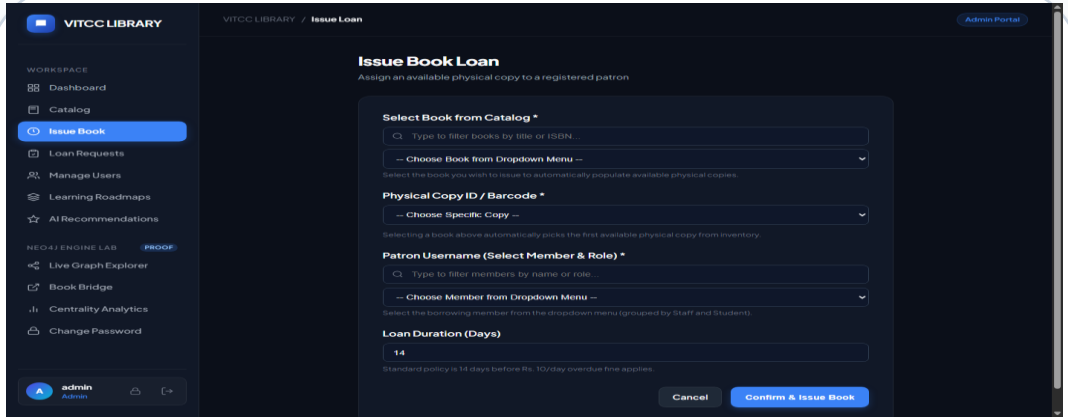
Working Application Screenshots — Circulation & Administration



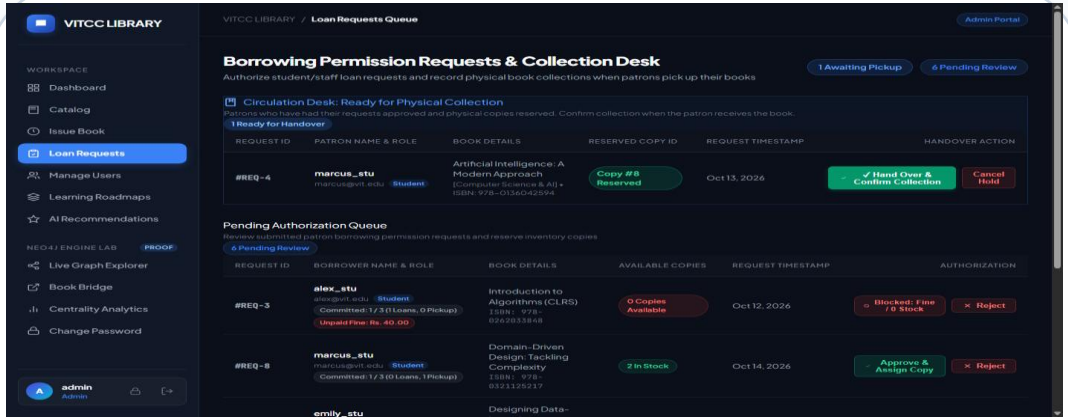
1. Secure Multi-Role Login Portal



2. Admin Dashboard & Active Circulation Ledgers

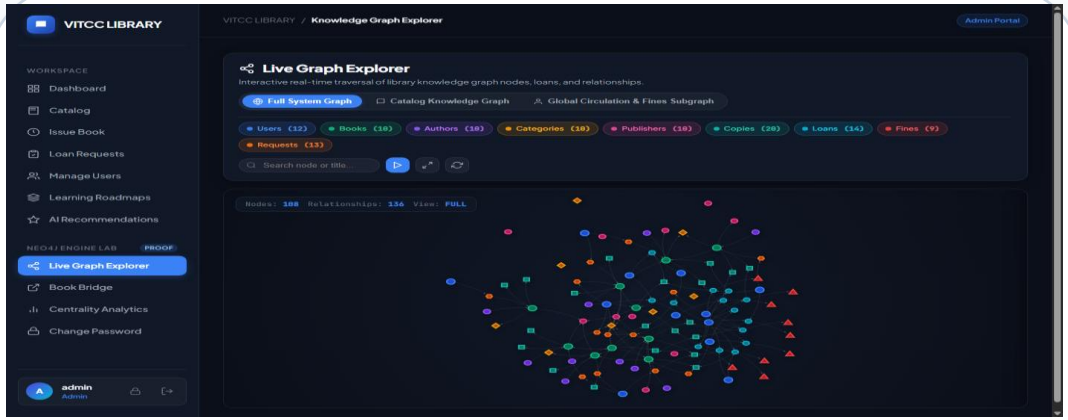


3. Physical Book Loan Issuance Desk

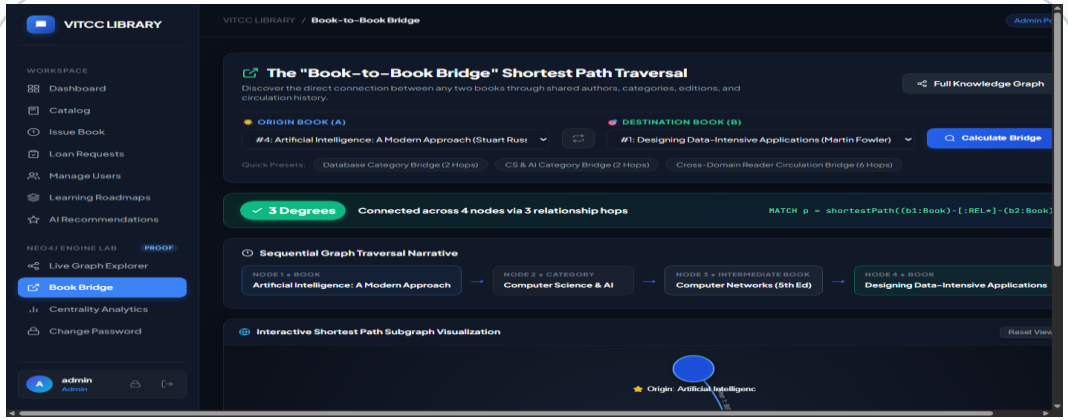


4. Digital Loan Requests & Copy Reservation Desk

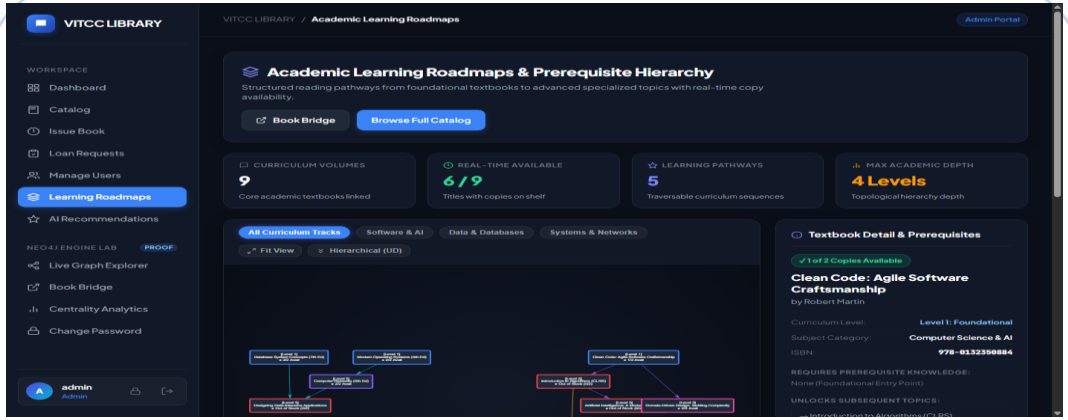
Working Application Screenshots — Graph & AI Innovations



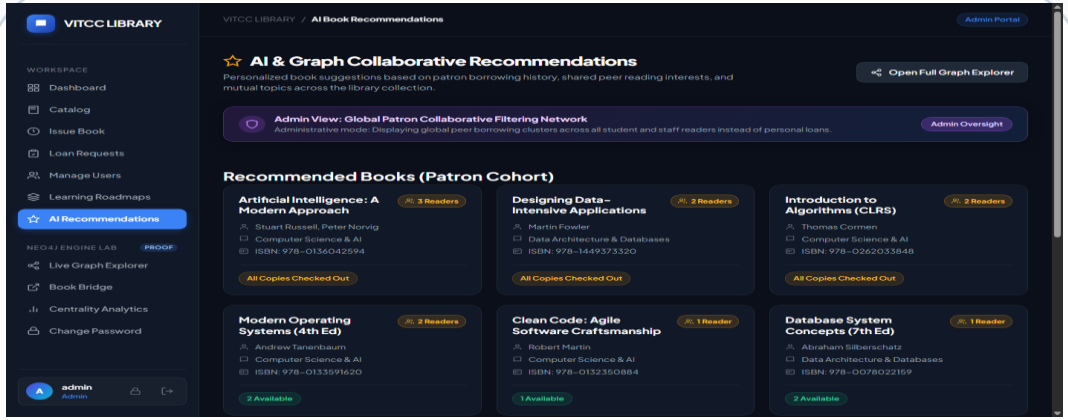
1. Live Knowledge Graph Explorer (/graph_visualizer)



2. Book Bridge Shortest Path Traversal (/book_bridge)



3. Academic Learning Roadmaps DAG (/learning_paths)



4. AI Collaborative Filtering Recommendations

Role-Based Access Control & Advanced Graph Innovations

ROLE-BASED ACCESS CONTROL (RBAC) MATRIX

ADMINISTRATOR

Full authority: User provisioning, role assignment, account suspension, password reset, book catalog registration and book deletion. Complete audit visibility.

LIBRARIAN

Circulation desk manager: Book loan creation, digital request approvals, copy reservations, check-in returns, and fine settlement. Cannot manage user accounts.

STAFF (FACULTY)

Academic borrower: Search catalog, borrow up to 3 books, view academic prerequisite roadmaps, explore author degree centrality, and review fine ledgers.

STUDENT (PATRON)

Academic patron: Search catalog, submit loan requests, view personalized AI recommendations, track prerequisite roadmaps, and monitor individual fines.

CUSTOM ARCHITECTURAL INNOVATIONS

1. Book-to-Book Bridge (Degrees of Separation)

Uses Cypher shortestPath() algorithm to uncover multi-hop links between disparate titles via prerequisites, co-authors, and categories.

2. Academic Learning Roadmaps (Directed Acyclic Graphs)

Hierarchical Vis.js network visualizer mapping foundational courses to advanced texts across CS, AI, and Distributed Systems.

3. Privacy-Preserving Patron Masking

When Students or Staff browse the Live Knowledge Graph, other patrons' identities and borrow requests are anonymized in real time.

4. Author Degree Centrality & Borrowing Density

Computes influential scholars by loan volume and category closeness centrality to inform faculty acquisition budgets.

Build, Setup & Execution Instructions

STEP 1: CLONE REPOSITORY & CREATE VIRTUAL ENVIRONMENT

```
git clone https://github.com/Karthi16-png/LIBRARAY-MANAGEMENT-SYSTEM.git
```

cd "DBS DA"

```
python -m venv .venv
```

.venv\Scripts\activate (Windows) OR source .venv/bin/activate (macOS/Linux)

STEP 2: INSTALL DEPENDENCIES & CONFIGURE ENVIRONMENT

```
pip install -r requirements.txt
```

Configure Neo4j AuraDB credentials in .env or environment variables:

NEO4J_URI=neo4j+s://<instance>.databases.neo4j.io

NEO4J_USERNAME=neo4j | NEO4J_PASSWORD=<password>

STEP 3: SEED GRAPH DATABASE (INITIAL SETUP)

```
python seed_all_data.py
```

-> Establishes 12 schema uniqueness constraints

-> Seeds 11 users across all 4 roles, 10 categories, 15+ books, copies, and prerequisite edges

STEP 4: LAUNCH PORTAL & RUN TEST SUITES

```
python app.py
```

-> Access interactive portal at http://127.0.0.1:5000/

Execute complete 7-phase automated verification runner:

```
python scratch/run_all_phases.py  (100% Passing)
```

GitHub Repository & Project Key Learning Outcomes

OPEN-SOURCE GITHUB REPOSITORY

Repository URL:
https://github.com/Karthi16-png/LIBRARAY-MANAGEMENT-SYSTEM

Production Codebase

Contains app.py (4,230 lines), seed scripts, 24 template views, and full verification test suites.

Branching & Hygiene

Clean Git commit history on branch main with .gitignore preventing credential exposure and cache staging.

Documentation & Dossier

Includes detailed README.md setup instructions and the 211-page master forensic audit report PDF.

PROJECT OUTCOMES & FUTURE ROADMAP

Database Migration Mastery

Gained deep architectural expertise by successfully migrating a transactional relational schema (Oracle 21c) into a modern, highly-connected graph store (Neo4j AuraDB).

PL/SQL & Constraint Automation

Successfully replaced error-prone application validation with autonomous database triggers, procedures, and check constraints.

Enterprise RBAC Implementation

Engineered strict multi-role permission separation with patron privacy masking and audit trail logging.

Future Roadmap 1: RFID Hardware Integration

Deploy physical RFID return drop-boxes for automated instantaneous check-in without librarian intervention.

Future Roadmap 2: SMS & Webhook Alerts

Automated email and WhatsApp notifications for books nearing return deadlines to minimize disciplinary fines.

THANK YOU!

Questions & Faculty Evaluation Discussion

VITCC Enterprise Library Management System
Dual Relational & Graph Database Implementation

Course: BACSE202 — Database Systems | Slot: F1+TF1
Faculty Guide: Dr. R. Priyadarshini

Team Members: KARTHIK VALLIAPPAN C (25BAI1051) | NITHIN RANGASWAMY (25BAI1210) | NATRAJ (25BAI1079)
School of Computer Science & Engineering (SCOPE), VIT Chennai